

CHAPTER 1 · FOUNDATIONS

Generative AI

GenAI Engineer Interview Handbook — Interview-Ready Deep Dive

1. Introduction

Definition

Generative AI (GenAI) refers to a class of machine learning systems that **create new content** — text, code, images, audio, video, or structured data — rather than simply classifying, scoring, or predicting a label on existing data. Formally, a generative model learns the underlying probability distribution $P(x)$ (or a conditional $P(x|y)$) of the training data and then samples from that distribution to produce novel, coherent outputs.

Why It Exists

Classical/discriminative ML answers the question "*what is this?*" (classification, regression). Businesses increasingly need systems that answer "*create this for me*" — draft an email, write code, summarize a contract, generate a product image. Generative AI exists to automate the creation of content and decisions that previously required a human expert to produce from scratch, at a fraction of the marginal cost and time.

Real-World Motivation

- Content creation at scale (marketing copy, code, documentation).
- Natural-language interfaces to software (chat-driven UX replacing forms/menus).
- Automating knowledge work: summarization, drafting, research synthesis.
- Personalization at a level impractical with hand-written rules.
- Powering autonomous agents that plan and execute multi-step tasks.

Historical Evolution

- **2013–2017:** Word2Vec/GloVe embeddings, RNNs/LSTMs for sequence generation, GANs (2014) for image synthesis.
- **2017:** "Attention Is All You Need" introduces the Transformer — the architectural backbone of modern GenAI.
- **2018–2020:** BERT (encoder-only, understanding), GPT-2/GPT-3 (decoder-only, generation) demonstrate emergent capability from scale.
- **2022:** ChatGPT (RLHF-tuned GPT-3.5) brings conversational LLMs to mainstream adoption; Stable Diffusion/DALL·E popularize text-to-image.
- **2023–2024:** GPT-4, Claude, Gemini — multimodal, long-context, tool-using models. Rise of RAG and agentic patterns to ground models in real, current, private data.
- **2025–2026:** Reasoning-optimized models (extended "thinking"/chain-of-thought at inference time), native agentic tool-use, and standardized interop protocols (e.g., MCP) for connecting models to external systems.

2. Core Concepts

Generative vs. Discriminative Models

A **discriminative** model learns $P(y|x)$ — the boundary between classes (e.g., spam vs. not-spam). A **generative** model learns $P(x)$ or $P(x|y)$ — the full data distribution — enabling it to sample new, unseen instances that resemble the training distribution.

Key Terminology

- **Model:** a parameterized function (billions of parameters in modern LLMs) trained to approximate a data distribution.
- **Parameters / Weights:** learned numeric values that encode what the model has learned from data.
- **Inference:** the process of running a trained model on new input to produce output (as opposed to training).
- **Sampling:** the stochastic process of drawing an output from the model's predicted probability distribution (controlled via temperature, top-p, top-k).
- **Modality:** the type of data a model handles — text, image, audio, video, or multimodal combinations.
- **Foundation Model:** a large model pretrained on broad data, later adapted (via prompting, fine-tuning, or RAG) to specific tasks.
- **Emergent Capability:** a qualitative skill (e.g., multi-step reasoning) that appears only past a certain scale threshold and isn't present in smaller versions of the same architecture.

The GenAI Model Lifecycle

- **1. Pretraining** — self-supervised learning on massive, broad corpora (next-token prediction for text).
- **2. Alignment / Post-training** — Supervised Fine-Tuning (SFT) on curated instruction data, then RLHF/DPO to align outputs with human preference (helpful, harmless, honest).
- **3. Adaptation** — prompting, RAG, or task-specific fine-tuning to specialize a general model for a domain.
- **4. Deployment / Inference** — serving the model behind an API, optimized for latency, throughput, and cost.
- **5. Evaluation & Monitoring** — offline benchmarks plus online quality/safety monitoring in production.

Design Decision: Model Families

Architecture	Typical Use	Example
Encoder-only	Understanding/classification, embeddings	BERT, sentence-transformers
Decoder-only	Text generation, chat, agents	GPT-4, Claude, Gemini, Llama
Encoder-Decoder	Translation, summarization (seq2seq)	T5, BART
Diffusion	Image/audio/video generation	Stable Diffusion, Sora

3. Internal Working

At the highest level, a modern generative language model works as an **autoregressive next-token predictor**: given a sequence of tokens, it outputs a probability distribution over the next token, samples one, appends it, and repeats.

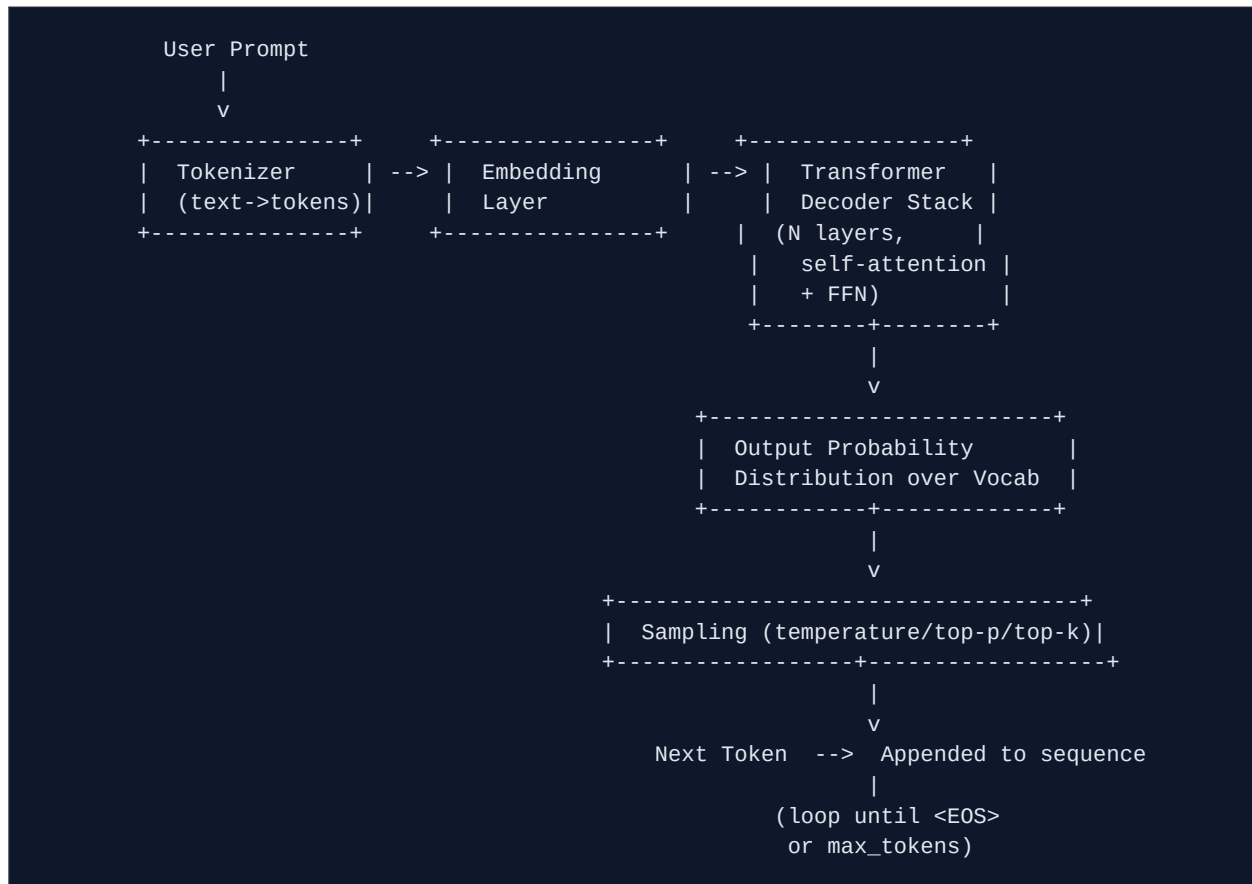


Figure 1 — Autoregressive text-generation loop (decoder-only LLM).

Execution Flow, Step by Step

- **1. Tokenization:** raw text is split into subword tokens using a learned vocabulary (e.g., BPE / SentencePiece).
- **2. Embedding:** each token ID is mapped to a dense vector; positional information is added (positional embeddings or rotary embeddings).
- **3. Transformer blocks:** stacked layers of self-attention (each token attends to relevant prior tokens) and feed-forward networks refine the representation.
- **4. Output projection:** the final hidden state is projected to vocabulary size and passed through softmax to get next-token probabilities.
- **5. Sampling / decoding:** a decoding strategy (greedy, beam search, top-k, nucleus/top-p) selects the next token.
- **6. Loop:** the new token is appended to the context and the process repeats until an end-of-sequence token or max-token limit is reached.

Interview angle: Being able to walk through this loop unprompted — including where temperature/top-p intervene — is a strong signal of real understanding vs. memorized vocabulary. Junior candidates say "it predicts the next word"; senior candidates can explain *where* stochasticity enters and *why* it matters for output diversity and determinism.

4. Production Perspective

How Companies Use It

- **Customer support:** chatbots/co-pilots that draft or fully automate responses (Intercom Fin, Zendesk AI).
- **Software engineering:** code completion and generation (GitHub Copilot, Cursor).
- **Content & marketing:** ad copy, product descriptions, SEO content generation at scale.
- **Knowledge work:** document summarization, meeting notes, research synthesis (Notion AI, Glean).
- **Search:** answer synthesis over retrieved sources (Perplexity, Google AI Overviews).

Scalability & Latency

Inference cost scales roughly with model size and output length. Production systems manage this via: model routing (small/cheap model for easy queries, large model for hard ones), response streaming (perceived latency reduction), caching of common prompts/responses, and batching requests on the server side to maximize GPU utilization.

Cost Optimization

- Prompt compression / trimming unnecessary context to reduce input token cost.
- Using smaller/distilled models for high-volume, low-complexity tasks.
- Caching (exact-match and semantic caching) to avoid redundant LLM calls.
- Batch APIs (async, discounted) for non-real-time workloads.
- Setting max_tokens deliberately to avoid paying for runaway generations.

Monitoring & Deployment

- Track latency (p50/p95/p99), token usage/cost per request, and error/timeout rates.
- Track output quality signals: hallucination rate, user thumbs-up/down, refusal rate.
- Canary/blue-green rollouts when switching model versions — behavior can shift subtly between versions.
- Guardrail layers (input/output moderation) deployed as a separate, fast pre/post-processing step.

5. Advantages

- Dramatically reduces time-to-first-draft for text, code, and creative content.
- General-purpose: one foundation model serves many tasks via prompting, without task-specific training.
- Natural-language interface lowers the barrier to using complex systems (no need to learn a query language).
- Can synthesize/summarize information across large volumes of unstructured data quickly.
- Improves consistently with scale and better post-training, without core architecture changes.

6. Limitations

- **Hallucination:** models can generate fluent, confident, but factually incorrect content.
- **No inherent grounding:** a base LLM only "knows" what was in training data up to its cutoff, unless augmented with retrieval or tools.

- **Non-determinism:** sampling-based generation means outputs can vary run-to-run unless temperature=0 and the provider guarantees determinism (many don't, fully).
- **Cost and latency** scale with model size and context length, which can conflict with real-time UX needs.
- **Bias and safety risk:** models can reproduce biases present in training data and can be misused for disinformation or harmful content without proper guardrails.
- **Context window limits:** even "long context" models degrade in retrieval accuracy over very long inputs ("lost in the middle" effect).

7. Best Practices

- Treat the LLM as a probabilistic component in a larger system — validate/guard its output, don't trust it blindly.
- Ground factual tasks with retrieval (RAG) rather than relying on parametric memory alone.
- Use structured output (JSON mode / function calling) wherever downstream code parses the response.
- Set explicit system prompts defining role, constraints, and output format — don't rely on defaults.
- Log prompts, outputs, and metadata for every production call to enable debugging and evaluation.
- Version-control prompts like code; regression-test them when switching model versions.

8. Real-World Examples

Product	GenAI Application
ChatGPT	General-purpose conversational assistant; plugins/tools for browsing, code execution.
Claude	Conversational assistant emphasizing long-context reasoning, coding, and safety-aligned behavior.
Gemini	Multimodal assistant integrated across Google Search, Workspace, and Android.
Microsoft Copilot	Embedded generation inside Office apps (Word, Excel, Outlook) and Windows.
Perplexity	Answer-engine that generates responses grounded in retrieved, cited web sources.
Cursor / GitHub Copilot	In-IDE code generation, completion, and multi-file refactoring.
Notion AI	In-document drafting, summarization, and Q&A; over a workspace's content.

8.1. Comparison Table — Generative AI vs. Traditional (Discriminative) ML

Dimension	Generative AI	Traditional / Discriminative ML
Goal	Model $P(x) / P(x y)$; produce new content	Model $P(y x)$; predict a label/score
Output	Open-ended text, image, audio, code	Fixed class, number, or ranked list
Data need	Massive, broad, often self-supervised	Labeled, task-specific datasets
Adaptation	Prompting / RAG / light fine-tuning	Usually requires retraining per task

Dimension	Generative AI	Traditional / Discriminative ML
Evaluation	Harder — subjective, needs human/LLM judges	Easier — accuracy, F1, AUC etc.
Typical cost	Higher per-inference (large models)	Lower per-inference (smaller models)

9. Interview Questions

Beginner

Q: What is Generative AI, and how does it differ from traditional ML?

A: Generative AI models the full data distribution $P(x)$ (or $P(x|y)$) to produce new content, while traditional/discriminative ML models $P(y|x)$ to predict labels on existing inputs. GenAI outputs are open-ended (text, image, audio); traditional ML outputs are typically fixed classes or scores.

Q: What is a foundation model?

A: A large model pretrained on broad, diverse data that can be adapted to many downstream tasks via prompting, fine-tuning, or retrieval augmentation, rather than being trained from scratch for each task.

Intermediate

Q: Why do LLMs hallucinate, and how would you mitigate it in production?

A: Hallucination stems from the model generating the statistically most plausible continuation, not necessarily a verified fact — there's no built-in fact-checking step. Mitigations: retrieval-augmented generation to ground responses in real sources, lowering temperature for factual tasks, requiring citations, and adding a verification/self-consistency pass before returning an answer.

Q: Explain temperature, top-p, and top-k in sampling.

A: Temperature scales the logits before softmax — lower values sharpen the distribution (more deterministic, conservative), higher values flatten it (more diverse, riskier). Top-k restricts sampling to the k most likely tokens; top-p (nucleus sampling) restricts to the smallest set of tokens whose cumulative probability exceeds p. They're often combined.

Advanced

Q: Why does model quality often continue improving with scale, and where does that break down?

A: Scaling laws show predictable loss reduction with more parameters/data/compute, and some capabilities emerge non-linearly past thresholds (few-shot reasoning, instruction-following). It breaks down due to data availability limits, diminishing returns relative to compute/cost, and the fact that scale alone doesn't fix alignment, grounding, or factuality — those need post-training and system-level design (RAG, tools, guardrails).

Scenario-Based

Q: Your GenAI product occasionally returns confidently wrong answers to customers. Walk through your fix.

A: First isolate whether it's a knowledge gap (needs RAG/grounding) or a reasoning failure (needs prompt/chain-of-thought or a stronger model). Add retrieval with citations for factual domains, add a lightweight verifier step comparing the answer against retrieved sources, lower temperature, and add human-in-the-loop review for high-stakes responses. Instrument logging to measure hallucination rate before/after.

FAANG Style

Q: How would you decide whether a use case needs fine-tuning, RAG, or just better prompting?

A: Start with prompting — it's cheapest and fastest to iterate. Move to RAG when the task needs current, private, or large-volume factual knowledge the model doesn't have. Consider fine-tuning only when you need to change model *behavior/style/format* consistently, when RAG+prompting plateaus, or when you need to reduce latency/cost by baking instructions into weights rather than long prompts.

Startup Style

Q: You have a tiny team and a tight budget — how do you ship a GenAI feature responsibly and fast?

A: Use a hosted foundation model API rather than self-hosting; start with prompting only; add RAG only if grounding is genuinely needed; log everything from day one; add basic input/output moderation; keep a human review step for anything customer-facing until you have evaluation data to trust the system autonomously.

Production Debugging

Q: Response quality degraded after a silent model version upgrade from the provider. What do you check?

A: Compare outputs on a fixed regression test set (prompts + expected properties) between old and new behavior; check if the provider changed default sampling params or system prompt handling; check for prompt/template compatibility issues (e.g., the model now interprets instructions differently); consider pinning to a specific model version rather than a floating alias if the provider offers it.

10. Common Mistakes

- **Misconception:** "The model understands/knows things" — it produces statistically likely continuations; it has no persistent beliefs or verified knowledge store.

- **Interview mistake:** conflating Generative AI with LLMs specifically — GenAI also covers image/audio/video/structured-data generation (diffusion models, GANs, VAEs).
- **Implementation mistake:** not setting temperature=0 (or near it) for tasks needing consistency (e.g., structured data extraction), leading to flaky, hard-to-debug behavior.
- **Implementation mistake:** trusting raw model output for anything safety- or business-critical without a validation layer.
- **Interview mistake:** being unable to explain the difference between training-time knowledge and retrieval-time (RAG) knowledge when asked how to keep answers current.

11. Summary

Generative AI models learn a data distribution and sample new content from it, in contrast to discriminative models that predict labels on fixed inputs. Modern text-based GenAI is dominated by decoder-only Transformer architectures using autoregressive next-token prediction, shaped by a pretraining → alignment → adaptation → deployment lifecycle. In production, the core engineering challenges are grounding (avoiding hallucination), cost/latency management, and building validation/guardrail layers around an inherently probabilistic component.

12. Cheat Sheet

Concept	Key Fact
Generative vs Discriminative	GenAI models $P(x)/P(x y)$ to create content; discriminative models $P(y x)$ to predict labels.
Core loop	Tokenize → embed → transformer layers → next-token distribution → sample → repeat.
Temperature	Controls randomness; 0 = near-deterministic, higher = more diverse/risky.
Top-p / Top-k	Restrict sampling pool by cumulative probability (p) or fixed count (k).
Hallucination	Confident but false output; mitigate with RAG, lower temperature, verification.
Foundation model	Broadly pretrained model adapted via prompting, RAG, or fine-tuning.
Lifecycle	Pretrain → SFT/RLHF alignment → task adaptation → deploy → monitor/evaluate.
Production levers	Model routing, caching, streaming, batching, prompt trimming for cost/latency.
Key risk	Treating output as ground truth without a validation/guardrail layer.

End of Chapter 1 — Generative AI. Next: Chapter 2 — Large Language Models (LLMs).